

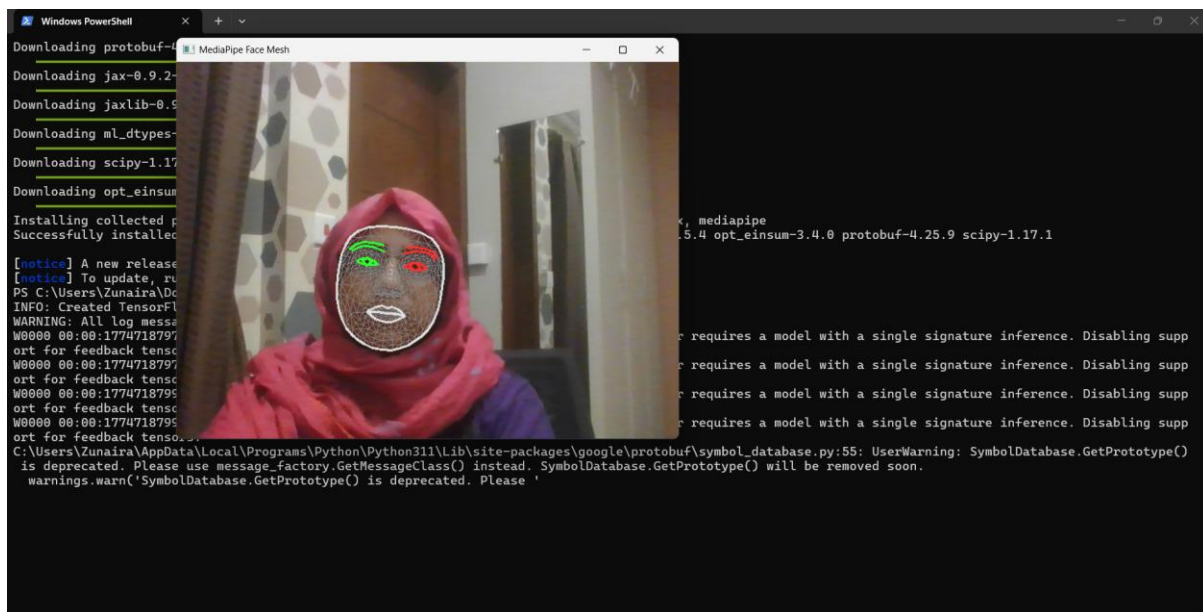


CLASS ACTIVITY-4: FACE MESH DETECTION USING MEDIAPIPE

Zunaira Abdul Aziz



BSSE23058-A



```
CODE: import cv2

import mediapipe as mp

# Correct imports (fix for your error)
from mediapipe.python.solutions import drawing_utils as mp_drawing
from mediapipe.python.solutions import drawing_styles as mp_drawing_styles
from mediapipe.python.solutions import face_mesh as mp_face_mesh

# =====
# Static image section
# =====

IMAGE_FILES = [] # Add image paths here if needed

drawing_spec = mp_drawing.DrawingSpec(thickness=1, circle_radius=1)

with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
) as face_mesh:

    for idx, file in enumerate(IMAGE_FILES):
        image = cv2.imread(file)

        if image is None:
            continue

        results = face_mesh.process(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

```

        if not results.multi_face_landmarks:
            continue

        annotated_image = image.copy()

        for face_landmarks in results.multi_face_landmarks:

            mp_drawing.draw_landmarks(
                image=annotated_image,
                landmark_list=face_landmarks,
                connections=mp_face_mesh.FACEMESH_TESSELATION,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing_styles.get_default_face_mes
h_tessellation_style(),
            )

            mp_drawing.draw_landmarks(
                image=annotated_image,
                landmark_list=face_landmarks,
                connections=mp_face_mesh.FACEMESH_CONTOURS,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing_styles.get_default_face_mes
h_contours_style(),
            )

            mp_drawing.draw_landmarks(
                image=annotated_image,
                landmark_list=face_landmarks,
                connections=mp_face_mesh.FACEMESH_IRISES,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing_styles.get_default_face_mes
h_iris_connections_style(),
            )

            cv2.imwrite("annotated_image_" + str(idx) + ".png", annotated_image)

# =====
# Webcam section
# =====

cap = cv2.VideoCapture(0)

with mp_face_mesh.FaceMesh(
    max_num_faces=1,
    refine_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5,
) as face_mesh:

```

```

while cap.isOpened():
    success, image = cap.read()

    if not success:
        print("Ignoring empty camera frame.")
        continue

    # Improve performance
    image.flags.writeable = False
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    results = face_mesh.process(image_rgb)

    image.flags.writeable = True
    image = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)

    if results.multi_face_landmarks:
        for face_landmarks in results.multi_face_landmarks:

            mp_drawing.draw_landmarks(
                image=image,
                landmark_list=face_landmarks,
                connections=mp_face_mesh.FACEMESH_TESSELATION,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing_styles.get_default_face
_mesh_tessellation_style(),
            )

            mp_drawing.draw_landmarks(
                image=image,
                landmark_list=face_landmarks,
                connections=mp_face_mesh.FACEMESH_CONTOURS,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing_styles.get_default_face
_mesh_contours_style(),
            )

            mp_drawing.draw_landmarks(
                image=image,
                landmark_list=face_landmarks,
                connections=mp_face_mesh.FACEMESH_IRISES,
                landmark_drawing_spec=None,
                connection_drawing_spec=mp_drawing_styles.get_default_face
_mesh_iris_connections_style(),
            )

    # Flip for selfie view

```

```
cv2.imshow("MediaPipe Face Mesh", cv2.flip(image, 1))

if cv2.waitKey(5) & 0xFF == 27: # ESC key
    break

cap.release()
cv2.destroyAllWindows()
```